

05: Kubernetes Operator Lab (GpuWorkload), Step by Step

The operator/ directory in this repository contains a working Kubernetes operator built with Kubebuilder and controller-runtime. It compiles, has unit tests, generates its CRD and RBAC manifests, and runs against a local Kind cluster.

This chapter walks through that operator in the order its code runs: the API types, the reconciler, the manager, the generated configuration, and the tests. It then shows how to run it, how to explain it in an interview, and how to extend it. 04-kubernetes-operator.md explains the underlying concepts; read it first if terms such as *reconcile* or *owner reference* are unfamiliar.

This chapter covers

- The GpuWorkload custom resource and the objects the operator creates from it
- API type design: required fields, pointer fields, defaults, and status
- The reconciler, step by step, including GPU resource requests and status updates
- Two defects in the reconciler and how to fix them
- The manager, RBAC markers, and unit tests with the fake client
- Running the operator on Kind with scripts/verify-kind.sh
- A summary you can give in an interview, and exercises that extend the lab

Contents

1. What the operator does.....	3
2. Project structure	4
3. The API types: api/v1/gpuworkload_types.go.....	5
3.1 The spec.....	5
3.2 The status	6
3.3 Type-level markers	6
4. The controller: internal/controller/gpuworkload_controller.go.....	7
4.1 Reconcile	7
4.2 SetupWithManager.....	8
4.3 reconcileDeployment	8
4.4 GPU resources	9
4.5 updateStatus.....	9
4.6 The phase and condition helpers.....	10
4.7 Known defects	10
5. The manager: cmd/main.go.....	11
6. RBAC markers and generated manifests	12
7. Unit tests: gpuworkload_controller_test.go	13

8. Running the operator on Kind.....	14
9. Explaining the operator in an interview	15
10. Exercises	16
Summary	17

1. What the operator does

The operator adds a GpuWorkload resource to the cluster. The sample for Kind, config/samples/gpucloud_v1_gpuworkload_kind.yaml, is:

```
apiVersion: gpucloud.example.com/v1
kind: GpuWorkload
metadata:
  name: triton-kind
  namespace: default
spec:
  image: nginx:1.27-alpine # CPU-safe sample for Kind
  replicas: 1
  gpuCount: 0 # local Kind nodes do not have GPUs
  servicePort: 8080
  containerPort: 80
```

For each GpuWorkload, the controller maintains three things:

Object	Name	Purpose
Deployment	<name>-gpu	Runs the container, with GPU requests when gpuCount is greater than 0
Service	<name>-svc	Exposes the container inside the cluster
The GpuWorkload status		Ready and available replica counts, a phase, and an Available condition

The second sample, config/samples/gpucloud_v1_gpuworkload_gpu.yaml, is for a real GPU cluster. It runs a Triton Inference Server image with gpuCount: 1, runtimeClassName: nvidia, a node selector for A100 nodes, and a toleration for the nvidia.com/gpu taint.

2. Project structure

```
operator/
├── PROJECT
├── Makefile
├── go.mod
├── cmd/main.go
├── api/v1/
│   ├── groupversion_info.go
│   ├── gpuworkload_types.go
│   └── zz_generated.deepcopy.go
├── internal/controller/
│   ├── gpuworkload_controller.go
│   └── gpuworkload_controller_test.go
├── config/
│   ├── crd/bases/
│   ├── rbac/
│   ├── samples/
│   └── manager/
└── Dockerfile
```

The structure separates three concerns:

- `api/v1/` defines the API contract that users write against.
- `internal/controller/` implements the behavior.
- `config/` holds manifests. `crd/bases/` and `rbac/` are generated from markers in the Go code; regenerate them with `make generate manifests` whenever you change a type or marker. `samples/` contains example resources, and `manager/` contains the operator's namespace.

3. The API types: api/v1/gpuworkload_types.go

3.1 The spec

```
type GpuWorkloadSpec struct {
    Image      string           `json:"image"`
    Replicas   *int32           `json:"replicas,omitempty"`
    GpuCount   int32            `json:"gpuCount,omitempty"`
    RuntimeClassName string         `json:"runtimeClassName,omitempty"`
    Command    []string         `json:"command,omitempty"`
    Args       []string         `json:"args,omitempty"`
    ServicePort int32           `json:"servicePort,omitempty"`
    ContainerPort int32          `json:"containerPort,omitempty"`
    Labels     map[string]string `json:"labels,omitempty"`
    NodeSelector map[string]string `json:"nodeSelector,omitempty"`
    Tolerations []Toleration      `json:"tolerations,omitempty"`
}
```

Markers on the fields add validation and defaults to the generated schema:

```
// +kubebuilder:validation:Required
Image string `json:"image"`

// +kubebuilder:validation:Minimum=0
// +kubebuilder:default=0
// +optional
GpuCount int32 `json:"gpuCount,omitempty"`
```

Each field reflects a design decision:

- Image **is required**. It has no omitempty tag and carries the Required marker, so the API server rejects a GpuWorkload without an image.
- Replicas **is a pointer**. A pointer distinguishes "not set" (nil) from an explicit 0. The controller uses 1 when the field is not set, and a user can still scale to zero.
- GpuCount **defaults to 0**. A CPU-only workload does not need to mention GPUs.
- ServicePort **and** ContainerPort **default to 8000** in the schema, the HTTP port of Triton Inference Server.
- Tolerations **uses a type defined in this package** rather than corev1.Toleration. The API exposes only the fields the operator supports, and its schema does not change when the Kubernetes core types change.

3.2 The status

```
type GpuWorkloadStatus struct {
    ReadyReplicas      int32      `json:"readyReplicas,omitempty"`
    AvailableReplicas  int32      `json:"availableReplicas,omitempty"`
    ObservedGeneration int64      `json:"observedGeneration,omitempty"`
    DeploymentName     string     `json:"deploymentName,omitempty"`
    ServiceName        string     `json:"serviceName,omitempty"`
    Phase              string     `json:"phase,omitempty"`
    Conditions          []metav1.Condition `json:"conditions,omitempty"`
}
```

The status subresource is enabled, so ordinary updates to the object cannot change the status, and only the controller, which has permission on `gpuworkloads/status`, writes it:

```
// +kubebuilder:subresource:status
```

3.3 Type-level markers

```
// +kubebuilder:object:root=true
// +kubebuilder:resource:path=gpuworkloads,scope=Namespaced,shortName=gw
// +kubebuilder:printcolumn:name="Phase",type=string,JSONPath=`.status.phase`
// +kubebuilder:printcolumn:name="GPUs",type=integer,JSONPath=`.spec.gpuCount`
// +kubebuilder:printcolumn:name="Ready",type=integer,JSONPath=`.status.readyReplicas`
```

The `printcolumn` markers add the phase, GPU count, and ready replicas to the output of `kubectl get gpuworkloads` (or `kubectl get gw`).

4. The controller: internal/controller/gpuworkload_controller.go

4.1 Reconcile

```
func (r *GpuWorkloadReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    logger := log.FromContext(ctx).WithValues("gpuworkload", req.NamespacedName)

    workload := &gpucloudv1.GpuWorkload{}
    if err := r.Get(ctx, req.NamespacedName, workload); err != nil {
        if apierrors.IsNotFound(err) {
            return ctrl.Result{}, nil
        }
        return ctrl.Result{}, err
    }

    if err := r.reconcileDeployment(ctx, workload); err != nil {
        logger.Error(err, "failed to reconcile Deployment")
        return ctrl.Result{}, err
    }

    if err := r.reconcileService(ctx, workload); err != nil {
        logger.Error(err, "failed to reconcile Service")
        return ctrl.Result{}, err
    }

    if err := r.updateStatus(ctx, workload); err != nil {
        logger.Error(err, "failed to update status")
        return ctrl.Result{}, err
    }

    return ctrl.Result{}, nil
}
```

The function follows the standard reconciler pattern:

1. **It reads the object by name.** The request contains only the namespace and name.
2. **It returns success if the object no longer exists.** The Deployment and Service have owner references to the GpuWorkload, so the garbage collector deletes them. The controller does not need to.
3. **It reconciles each child in turn.** Any error is returned, and the work queue retries the request with exponential backoff.
4. **It updates the status last,** after the children reflect the current spec.

4.2 SetupWithManager

```
func (r *GpuWorkloadReconciler) SetupWithManager(mgr ctrl.Manager) error {
    return ctrl.NewControllerManagedBy(mgr).
        For(&gpucloudv1.GpuWorkload{}).
        Owns(&appsv1.Deployment{}).
        Owns(&corev1.Service{}).
        Complete(r)
}
```

- For reconciles a GpuWorkload whenever it changes.
- Owns maps changes to a Deployment or Service back to the GpuWorkload named in its controller owner reference. When the Deployment's availableReplicas changes, this watch triggers a reconcile, and the controller updates the status.
- The controller does not watch nodes. Adding a Watches on Node objects would let it react when GPU nodes are added or removed, without any change to a GpuWorkload.

4.3 reconcileDeployment

```
deployment := &appsv1.Deployment{
    ObjectMeta: metav1.ObjectMeta{
        Name:      deploymentName(workload.Name),
        Namespace: workload.Namespace,
    },
}

_, err := controllerutil.CreateOrUpdate(ctx, r.Client, deployment, func() error {
    if err := controllerutil.SetControllerReference(workload, deployment, r.Scheme); err != nil {
        return err
    }
    // Set labels, replicas, image, ports, GPU resources, runtime class.
    return nil
})
```

CreateOrUpdate reads the Deployment, applies the mutate function, and then creates the Deployment if it did not exist or updates it if the mutate function changed it. Inside the mutate function, the controller sets the owner reference, merges the user's labels with the operator's own labels, and fills in the Deployment spec: the replica count, the Pod template with the image, command, arguments, runtime class, node selector, tolerations, container port, and GPU resources.

Three details of the mutate function are important:

- **The owner reference is set in the mutate function**, so it is present when the Deployment is first created. A child created without it would not be garbage collected with its GpuWorkload, and its events would not reach this controller.
- **The Service preserves fields that the API server assigns.** reconcileService copies ClusterIP, ClusterIPs, IPFamilies, and IPFamilyPolicy from the existing

Service before replacing its spec. Those fields are immutable after creation, and clearing them would make every update fail.

- **The Deployment spec is replaced as a whole.** See the first defect in section 4.7.

4.4 GPU resources

```
func gpuResources(gpuCount int32) corev1.ResourceRequirements {
    if gpuCount == 0 {
        return corev1.ResourceRequirements{}
    }
    quantity := resource.MustParse(strconv.Itoa(int(gpuCount)))
    return corev1.ResourceRequirements{
        Requests: corev1.ResourceList{"nvidia.com/gpu": quantity},
        Limits:   corev1.ResourceList{"nvidia.com/gpu": quantity},
    }
}
```

nvidia.com/gpu is an extended resource. Kubernetes requires extended resources in limits, and when requests is also set, the two must be equal. The function sets both to the same quantity, so the Pod template contains:

```
resources:
  requests:
    nvidia.com/gpu: "1"
  limits:
    nvidia.com/gpu: "1"
```

For a CPU-only workload, the function returns empty requirements, and the Pod can run on nodes without a GPU device plugin, such as the nodes of a Kind cluster.

4.5 updateStatus

updateStatus reads the Deployment, takes its ReadyReplicas and AvailableReplicas, builds a new status, and writes it only if it differs from the current status:

```
latest := &gpucloudv1.GpuWorkload{}
if err := r.Get(ctx, client.ObjectKeyFromObject(workload), latest); err != nil {
    return err
}
newStatus := ...
if equality.Semantic.DeepEqual(latest.Status, newStatus) {
    return nil
}
latest.Status = newStatus
return r.Status().Update(ctx, latest)
```

The function reads the GpuWorkload again before writing. The object read at the start of Reconcile may be out of date by this point, and an update with an old resourceVersion fails with a conflict. Reading the latest version reduces conflicts, and the status's ObservedGeneration is taken from that latest object. A client compares

`status.observedGeneration` with `metadata.generation` to check whether the status reflects the current spec.

4.6 The phase and condition helpers

`phaseFor` and `conditionsFor` are pure functions: they take numbers and return values, without calling the API. Keeping them separate makes them easy to unit test and keeps `Reconcile` short.

Desired replicas	Ready	Available	phaseFor result
0	any	any	""
2	2	2	Ready
3	2	2	Degraded
3	0	0	Pending

`conditionsFor` returns one `Available` condition, with reason `DeploymentAvailable` when all desired replicas are available and `NotAvailable` otherwise.

4.7 Known defects

The operator is intentionally small, and it has two defects that are worth understanding and fixing.

The Deployment selector includes user labels. `reconcileDeployment` uses the merged labels, including `spec.labels` from the user, for both the Pod template and `spec.selector`. A Deployment's selector cannot change after creation. If a user edits `spec.labels` on an existing `GpuWorkload`, every later update of the Deployment is rejected, and the error repeats on each retry. The fix is to build the selector only from the labels the operator controls, `managedByLabel` and `workloadLabel`, and to apply the user's labels only to the Pod template and the Deployment's metadata.

`LastTransitionTime` **is reset on every reconcile.** `conditionsFor` sets `LastTransitionTime` to the current time each time it runs. The timestamp should change only when the condition's status changes. Because of the new timestamp, the equality check in `updateStatus` almost never succeeds, so the controller writes the status on nearly every reconcile. The fix is to start from the existing conditions and call `meta.SetStatusCondition`, which keeps the previous timestamp when the status is unchanged.

A related inefficiency: `reconcileDeployment` replaces the whole Deployment spec, including fields the API server fills with defaults, such as the rollout strategy. The mutated object therefore differs from the stored one on each reconcile, and `CreateOrUpdate` sends an update every time. Setting only the fields the operator owns, or using server-side apply, avoids those requests.

5. The manager: cmd/main.go

cmd/main.go starts the operator:

1. It builds a scheme containing the client-go types and the gpucloud/v1 types.
2. It creates a manager with the metrics address, health probe address, and leader election options.
3. It registers the reconciler with SetupWithManager.
4. It adds /healthz and /readyz checks.
5. It starts the manager, which runs until it receives SIGTERM or SIGINT.

Details to note:

- LeaderElectionID: "gpu-cloud-operator.example.com" names the Lease object used for leader election. It must be unique among the operators in a namespace.
- **The metrics and health probe addresses are flags**, so the CI job and the Kind script can run the operator on ports that do not conflict with other processes.
- ctrl.GetConfigOrDie() uses the in-cluster service account configuration when the operator runs in a Pod, and the KUBECONFIG environment variable or ~/.kube/config when it runs on your machine.

6. RBAC markers and generated manifests

The controller declares the permissions it needs:

```
//  
+kubebuilder:rbac:groups=gpucloud.example.com,resources=gpuworkloads,verbs=get;list;watch;create;update;patch;delete  
// +kubebuilder:rbac:groups=gpucloud.example.com,resources=gpuworkloads/status,verbs=get;update;patch  
// +kubebuilder:rbac:groups=gpucloud.example.com,resources=gpuworkloads/finalizers,verbs=update  
// +kubebuilder:rbac:groups=apps,resources=deployments,verbs=get;list;watch;create;update;patch;delete  
// +kubebuilder:rbac:groups=core,resources=services,verbs=get;list;watch;create;update;patch;delete  
// +kubebuilder:rbac:groups=core,resources=pods,verbs=get;list;watch
```

make manifests writes these rules to config/rbac/role.yaml as a ClusterRole. Permissions on the status and finalizers subresources are listed separately because the API server authorizes each subresource independently. Permission to update gpuworkloads does not include permission to update gpuworkloads/status.

7. Unit tests: gpuworkload_controller_test.go

The tests run the reconciler against controller-runtime's in-memory fake client:

```

scheme := runtime.NewScheme()
utilruntime.Must(clientgoscheme.AddToScheme(scheme))
utilruntime.Must(gpucloudv1.AddToScheme(scheme))

c := fake.NewClientBuilder().
    WithScheme(scheme).
    WithObjects(workload).
    WithStatusSubresource(&gpucloudv1.GpuWorkload{}).
    Build()

```

The two tests are:

Test	What it verifies
TestReconcileCreatesOwnedDeploymentServiceAndStatus	The Deployment and Service are created with an owner reference, the runtime class and <code>nvidia.com/gpu</code> resources are set for a GPU workload, and the status records the child names and a phase
TestReconcileCpuWorkloadDoesNotRequestGPU	A workload with <code>gpuCount: 0</code> produces a Pod template without <code>nvidia.com/gpu</code>

The fake client has limits. It does not run the Deployment controller, so replicas never become ready. It does not validate objects against the CRD schema. It does not exercise the watches and work queue. It is well suited to testing reconcile logic, the objects the controller builds, and status subresource behavior. Section 10 suggests adding envtest to cover the rest.

8. Running the operator on Kind

```
scripts/verify-kind.sh
```

The script performs these steps:

1. Checks that go, kind, and kubectl are installed.
2. Creates a Kind cluster named systems-interview, unless it already exists.
3. Installs the CRD and waits for it to be Established.
4. Runs `go test ./...`
5. Builds the operator and starts it in the background.
6. Waits for the operator's /healthz endpoint to respond.
7. Applies the Kind sample.
8. Waits for the triton-kind-gpu Deployment to finish rolling out, and for the GpuWorkload status to report readiness.
9. Prints the GpuWorkload, Deployment, Service, Pods, and operator log.

The Kind sample sets `gpuCount: 0`, because Kind nodes have no GPU driver or device plugin. A Pod that requested `nvidia.com/gpu` would stay Pending. On a real GPU cluster with the GPU Operator installed, apply the GPU sample instead.

9. Explaining the operator in an interview

A concise description of the design:

"The operator adds a GpuWorkload CRD with a small declarative spec. The controller watches GpuWorkload resources and the Deployments and Services they own. On each reconcile, it reads the latest GpuWorkload, creates or updates a Deployment that requests `nvidia.com/gpu` when the workload needs GPUs, creates or updates a Service, and then copies the Deployment's replica counts into the status, with a condition and `observedGeneration`. The children have controller owner references, so Kubernetes deletes them when the GpuWorkload is deleted. The CRD and RBAC are generated from markers. Unit tests use the fake client, and an end-to-end test runs on Kind."

Be ready for these follow-up questions:

Question	Answer
How would you add a finalizer?	Add the finalizer before creating any external resource. At the start of Reconcile, check <code>DeletionTimestamp</code> ; if it is set, clean up the external resource, remove the finalizer, and return before reconciling the children
How would you reject <code>gpuCount</code> above 8?	Add <code>+kubebuilder:validation:Maximum=8</code> to the field. Use a CEL rule for conditions involving several fields, and a validating webhook for policy that depends on external data
How would you add custom metrics?	Register a Prometheus counter or histogram with the controller-runtime metrics registry and record values in the reconciler
What happens if the operator is deleted?	The Deployments and Services continue to run. Status stops updating, spec changes are not applied, and any resource with a finalizer cannot finish deleting
Why set owner references on the Deployment?	For garbage collection when the GpuWorkload is deleted, and so that <code>Owns</code> can map Deployment events to the right GpuWorkload

10. Exercises

1. Fix the two defects in section 4.7, and add tests that fail before the fix.
2. Add a **validating webhook** that rejects gpuCount above a configured maximum.
3. Add a **mutating webhook** that sets servicePort and containerPort from the known port of the image, for example 8000 for Triton.
4. Add a **finalizer** that releases a reservation in an external system and blocks deletion until the release succeeds.
5. Add a **watch on nodes** that sets a condition on workloads when no node matches their GPU node selector.
6. Add **Prometheus metrics** for reconcile duration, errors, and the number of workloads in each phase.
7. Add **envtest** integration tests that validate the generated CRD schema.
8. Add a v1alpha1 **version** and a conversion webhook to practice API versioning.
9. Enable **leader election** with --leader-elect, run two replicas, and stop the leader to observe the failover.

Summary

- The operator turns each `GpuWorkload` into a `Deployment` and a `Service`, and reports their state in the status.
- The API types use a required field, a pointer for an optional count, schema defaults, and a status subresource.
- The reconciler reads the latest object, uses `CreateOrUpdate` with owner references for each child, and writes the status only when it changes.
- GPU workloads request `nvidia.com/gpu` in both requests and limits with equal values.
- The selector built from user labels and the reset condition timestamp are defects to fix.
- Unit tests use the fake client, and `scripts/verify-kind.sh` runs the full flow on Kind.

Source: [05-kubernetes-operator-lab.md](#) · [PDF](#) · [All study guides](#)